

Πίνακες (Arrays)

Μιας Διάστασης

Πίνακες

- Οι πίνακες αποτελούν ένα σημαντικό **δομημένο τύπο δεδομένων** (structured data type) ή πιο απλά μια **δομή δεδομένων** (data structure)
- Οι απλοί τύποι δεδομένων, δηλαδή οι τύποι δεδομένων όπου οι τιμές τους δεν μπορούν να διασπαστούν σε απλούστερα στοιχεία, δεν αρκούν από μόνοι τους:
 - Για υψηλότερη αφαιρετικότητα (δεδομένων) χρειάζονται δομές δεδομένων

Μία **δομή δεδομένων** είναι μία συνάθροιση συγγενών δεδομένων τα οποία αποθηκεύονται κάτω από το ίδιο όνομα

Ένας **πίνακας** είναι μία δομή δεδομένων, όπου ένα σύνολο αντικειμένων του ίδιου τύπου αποθηκεύονται σε σειρά

Παραδείγματα

- 1 εκατομμύριο χαρακτήρες σε ένα βιβλίο
- 10 εκατομμύρια δείγματα ήχου σε ένα αρχείο MP3
- 73 δισεκατομμύρια αναζητήσεις Google το χρόνο
- 50 τρισεκατομμύρια κύτταρα στο ανθρώπινο σώμα
- 6 χιλιάδες φοιτητές στο Πανεπιστήμιο Κύπρου
- 52 χαρτιά στην τράπουλα
- κτλ.

Πίνακας
MonthDays

index	value
0	31
1	28
2	31
3	30
4	31
5	30
6	31
7	31
8	30
9	31
10	30
11	31

Πολλές Μεταβλητές του Ιδίου Τύπου

- Στόχος: 10 μεταβλητές του ιδίου τύπου

```
// tedious and error-prone
double a0, a1, a2, a3, a4, a5, a6, a7, a8, a9;
a0 = 0.0;
a1 = 0.0;
a2 = 0.0;
a3 = 0.0;
a4 = 0.0;
a5 = 0.0;
a6 = 0.0;
a7 = 0.0;
a9 = 0.0;
a9 = 0.0;

double x = a4 + a8;
```

Πίνακες στην Java

Η Java παρέχει ειδική στήριξη σε πίνακες:

1. Για να δημιουργηθεί ένας πίνακας πρέπει να

δηλωθεί, `double[] a;`

υλοποιηθεί και `a = new double[10];`

αρχικοποιηθεί `for (int i = 0; i < N; i++) a[i]=0.0;`

2. Αναφερόμαστε στο στοιχείο **i** ενός πίνακα **a** με την έκφραση **a[i]**

Ο τύπος της έκφρασης του i πρέπει να είναι διακριτός (int, char)

3. Η επεξεργασία ενός πίνακα γίνεται στοιχείο με στοιχείο

`a[i]=3; ++a[j+1]; c[m]=b[a[k]];`

4. Τα στοιχεία ενός πίνακα **με n στοιχεία** απαριθμούνται (είναι **indexed**) από το **0** μέχρι το **n-1**

```
int N = 10;
double[] a;           // declare the array
a = new double[N];    // create the array
for (int i = 0; i < N; i++) // initialize the array
    a[i] = 0.0;       // all to 0.0
```

Συμπαγής, εναλλακτική λύση

```
int N = 10;
double[] a = new double[N]; // declare, create, init
```

Default αρχικοποίηση: σε όλους τους αριθμούς ανατίθεται αυτόματα η τιμή 0. Όμως η ρητή αρχικοποίηση αποτελεί γενικά καλύτερη πρακτική.

Παράδειγμα: Vector Dot Product

Το άθροισμα των γινομένων αντίστοιχων στοιχείων δύο διανυσμάτων, $x[]$ και $y[]$

```
double[] x = { 0.3, 0.6, 0.1 };  
double[] y = { 0.5, 0.1, 0.4 };  
double sum = 0.0;  
for (int i = 0; i < N; i++) {  
    sum += x[i]*y[i];  
}
```

$$\sum_{i=0}^{n-1} x_i y_i$$

i	x[i]	y[i]	x[i]*y[i]	sum
				0
0	.30	.50	.15	.15
1	.60	.10	.06	.21
2	.10	.40	.04	.25
				.25

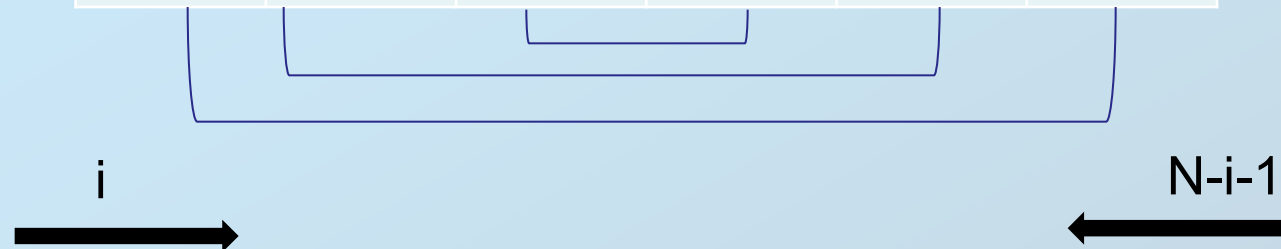
Παραδείγματα Επεξεργασίας Πινάκων

<i>create an array with random values</i>	<pre>double[] a = new double[N]; for (int i = 0; i < N; i++) a[i] = Math.random();</pre>
<i>print the array values, one per line</i>	<pre>for (int i = 0; i < N; i++) System.out.println(a[i]);</pre>
<i>find the maximum of the array values</i>	<pre>double max = Double.NEGATIVE_INFINITY; for (int i = 0; i < N; i++) if (a[i] > max) max = a[i];</pre>
<i>compute the average of the array values</i>	<pre>double sum = 0.0; for (int i = 0; i < N; i++) sum += a[i]; double average = sum / N;</pre>
<i>copy to another array</i>	<pre>double[] b = new double[N]; for (int i = 0; i < N; i++) b[i] = a[i];</pre>
<i>reverse the elements within an array</i>	<pre>for (int i = 0; i < N/2; i++) { double temp = b[i]; b[i] = b[N-1-i]; b[N-i-1] = temp; }</pre>

*reverse the elements
within an array*

```
for (int i = 0; i < N/2; i++)  
{  
    double temp = b[i];  
    b[i] = b[N-1-i];  
    b[N-i-1] = temp;  
}
```

0	1	2	3	4	5
4	5	-1	2	2	8



0	1	2	3	4	5
8	2	2	-1	5	4

```
for (int i = 0, j = N-1; i < j ; i++, j--){  
    double temp = b[i];  
    b[i] = b[j];  
    b[j] = temp;  
}
```

Επεξεργασία Πίνακα

Η επεξεργασία ενός πίνακα γίνεται στοιχείο με στοιχείο, π.χ.

```
System.out.println(a[0]);  
a[3] = 25.0;  
sum = a[0] + a[1];  
sum += a[2];  
a[3] += 1.0;  
a[2] = a[0] + a[1];
```

Τα στοιχεία ενός πίνακα αποθηκεύονται σε συνεχόμενες θυρίδες στη μνήμη του υπολογιστή. Ως εκ τούτου υπάρχει τυχαία προσπέλαση σε οποιοδήποτε στοιχείο του πίνακα. Αυτό είναι ένα από τα πλεονεκτήματα της δομής του πίνακα.

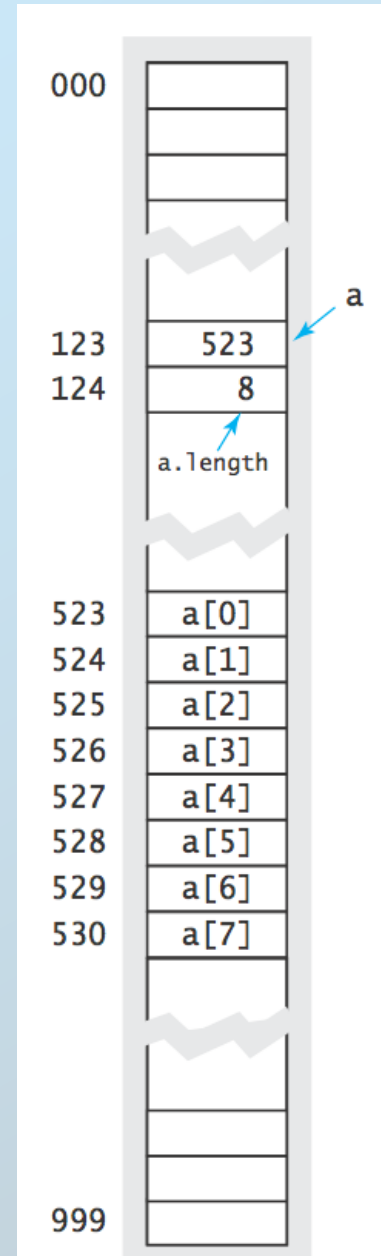
Αναπαράσταση Πίνακα στη Μνήμη

- Η αναπαράσταση ενός πίνακα στη μνήμη αντιστοιχεί απευθείας στη φυσική δομή του υπολογιστή

```
double []a = new double[8];
```

```
// το a δείχνει στην αρχή του πίνακα  
// a.length is 8, χαρακτηριστικό του  
πίνακα 8
```

- Συνέπειες:
 - Οι πίνακες έχουν σταθερό μέγεθος
 - Υπάρχει ταχεία πρόσβαση στα στοιχεία του πίνακα μέσω του index τους
 - Οι πίνακες είναι δείκτες



```
double[] x = {16.0,12.0,6.0,8.0,2.5,12.0, 14.0,-54.3} ;
```

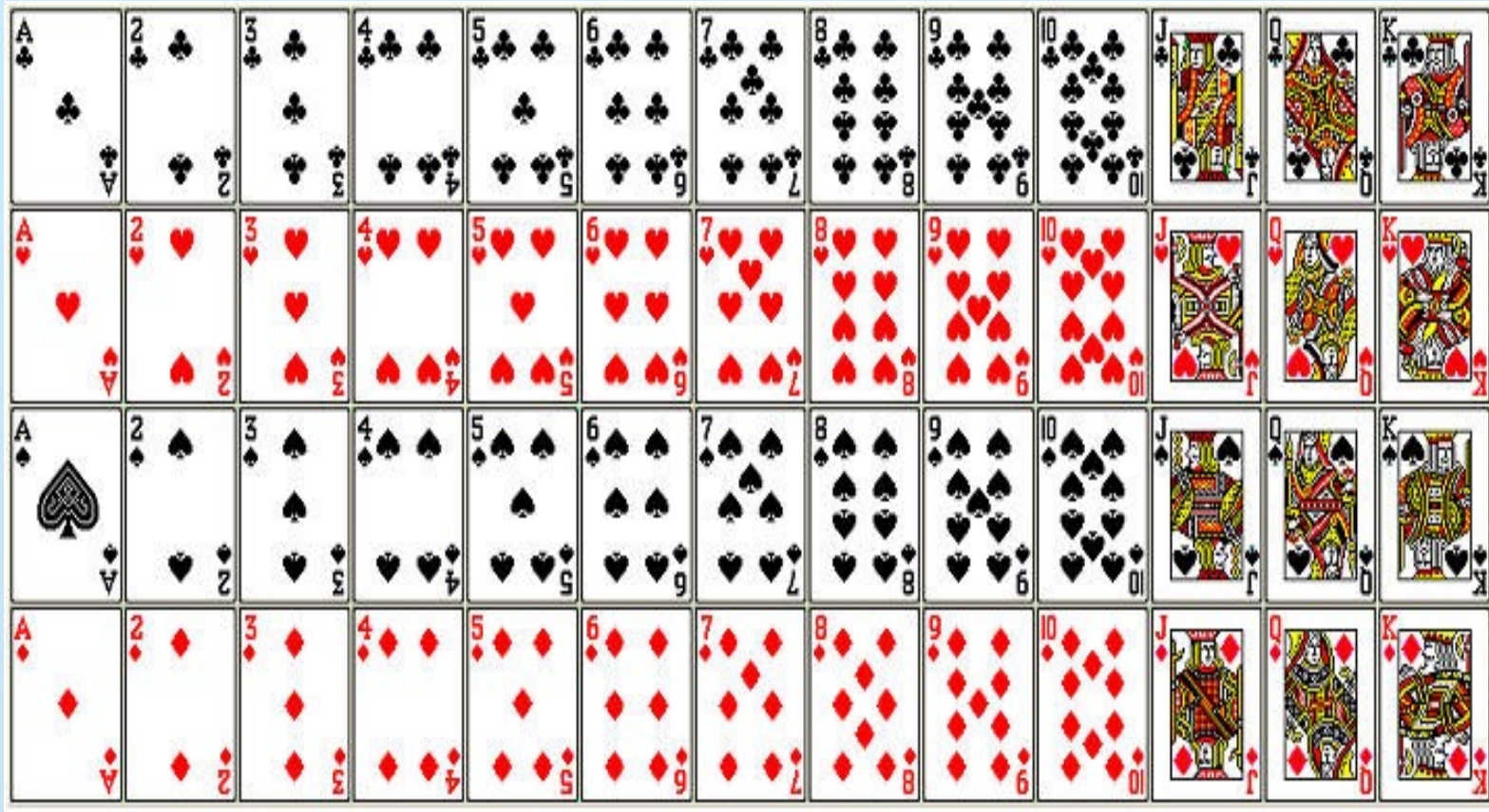
Πίνακας x

x[0]	x[1]	x[2]	x[3]	x[4]	x[5]	x[6]	x[7]
16.0	12.0	6.0	8.0	2.5	12.0	14.0	-54.5

Αναφορά σε στοιχεία του πίνακα: Τι σημαίνουν οι ακόλουθες εκφράσεις;

```
int i = 5;  
x[4]  
x[i]  
x[i+1]  
x[i+i]  
x[2*i]  
x[2*i-3]  
x[(int)x[4]]  
x[i++]  
x[--i]  
x[i-1] = x[i];  
x[i] = x[i+1];  
x[i] - 1 = x[i];
```

Παράδειγμα: Το ανακάτεμα της τράπουλας



$4 \times 13 = 52$ χαρτιά (4 φύλλα, 13 σειρές)

Πίνακας με 52 θέσεις και η κάθε θέση αναπαριστά ένα μοναδικό χαρτί με String

Παράδειγμα: Το ανακάτεμα της τράπουλας

Διάσπαση προβλήματος σε υπο-προβλήματα

Επιλογή και εκτύπωση ενός τυχαίου χαρτιού από την τράπουλα

```
String[] rank = {  
    "2", "3", "4", "5", "6", "7", "8", "9",  
    "10", "Jack", "Queen", "King", "Ace"  
};  
  
String[] suit = {  
    "Clubs", "Diamonds", "Hearts", "Spades"  
};  
  
int i = (int) (Math.random() * 13); // between 0 and 12  
int j = (int) (Math.random() * 4);  // between 0 and 3  
  
System.out.println(rank[i] + " of " + suit[j]);
```

Δημιουργία και εκτύπωση μιας τράπουλας

```
String[] deck = new String[rank.length*suit.length];  
for (int i = 0; i < suit.length; i++)  
    for (int j = 0; j < rank.length; j++)  
        deck[i*rank.length + j] = rank[j] + " of " + suit[i];  
  
for (int i = 0; i < 52; i++)  
    System.out.println(deck[i]);
```

Σε ποια σειρά εκτυπώνεται η τράπουλα;

A. 2 of clubs
 2 of diamonds
 2 of hearts
 2 of spades
 3 of clubs
 ...

B. 2 of clubs
 3 of clubs
 4 of clubs
 5 of clubs
 6 of clubs
 ...

Ανακάτεμα της τράπουλας

Στόχος είναι να μπουν τα στοιχεία του πίνακα που αντιπροσωπεύει την τράπουλα σε τυχαία σειρά

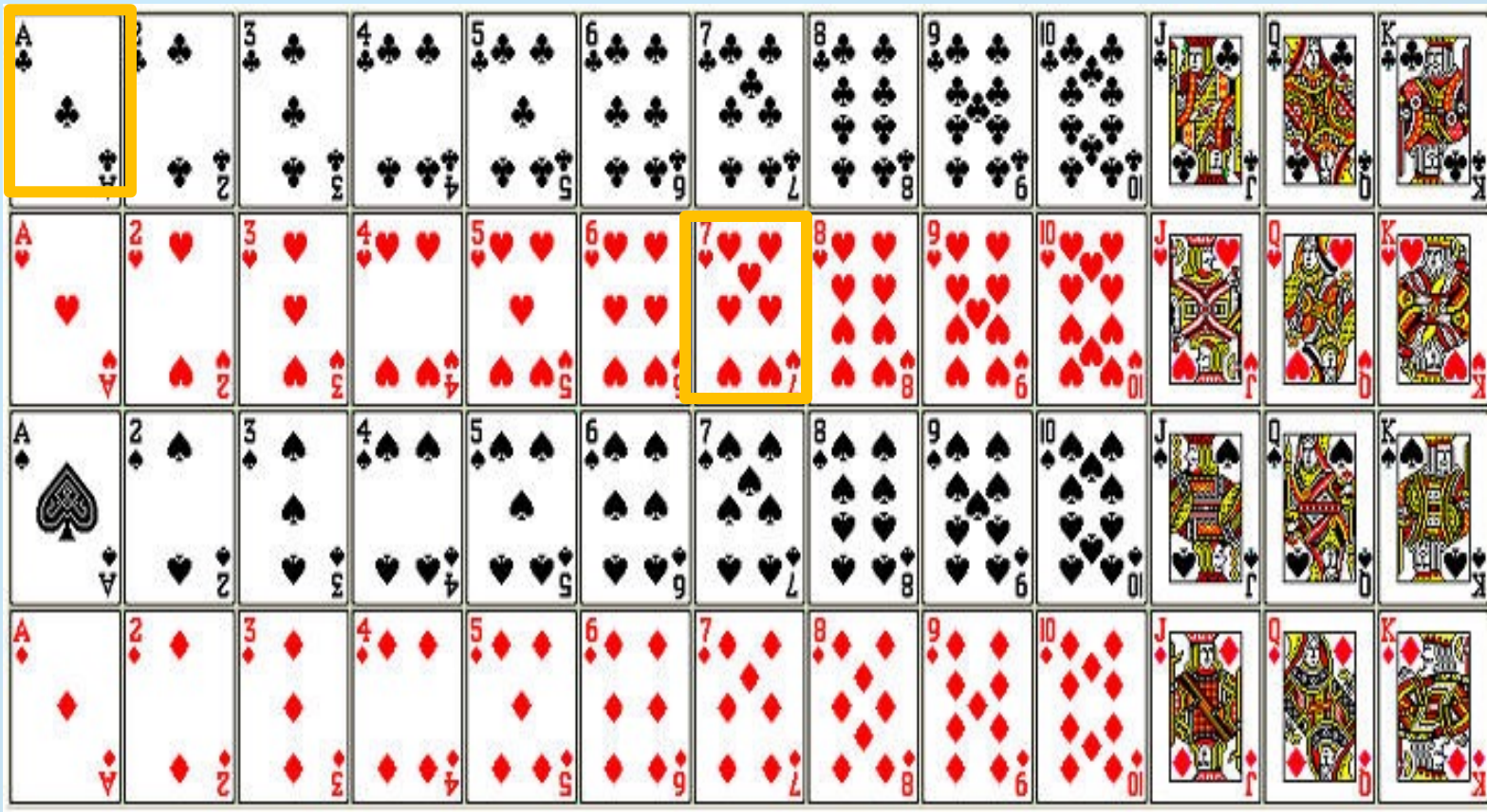
Αλγόριθμος για το Ανακάτεμα της Τράπουλας, `deck`:

Επανάλαβε όσες φορές είναι τα χαρτιά της τράπουλας (**`N`**)

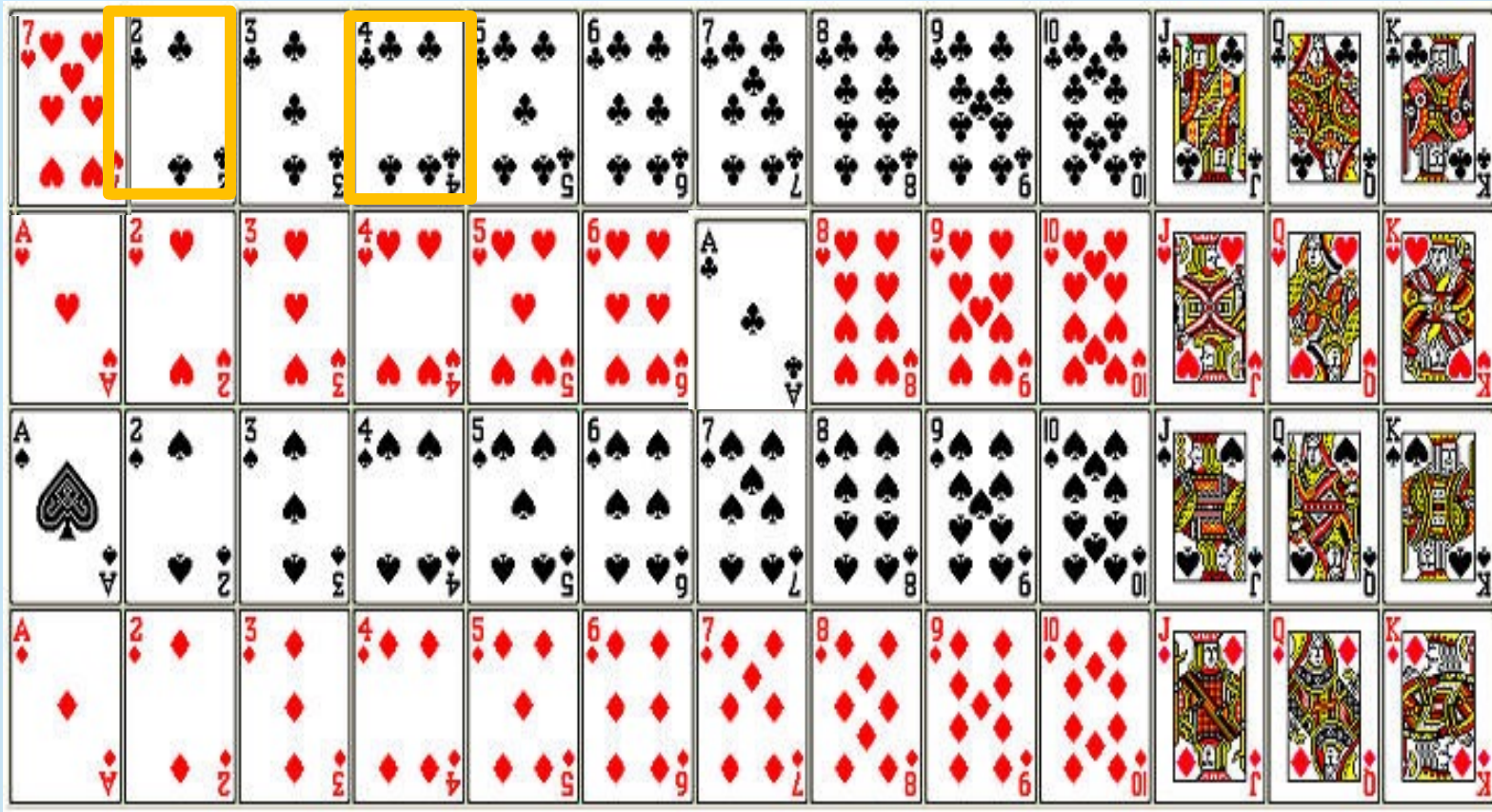
Σε κάθε επανάληψη **`i`**, επέλεξε ένα τυχαίο χαρτί από το τμήμα της τράπουλας **`deck[i]`** μέχρι **`deck[N-1]`**

Αντάλλαξε το χαρτί αυτό με εκείνο που βρίσκεται στη θέση **`deck[i]`**

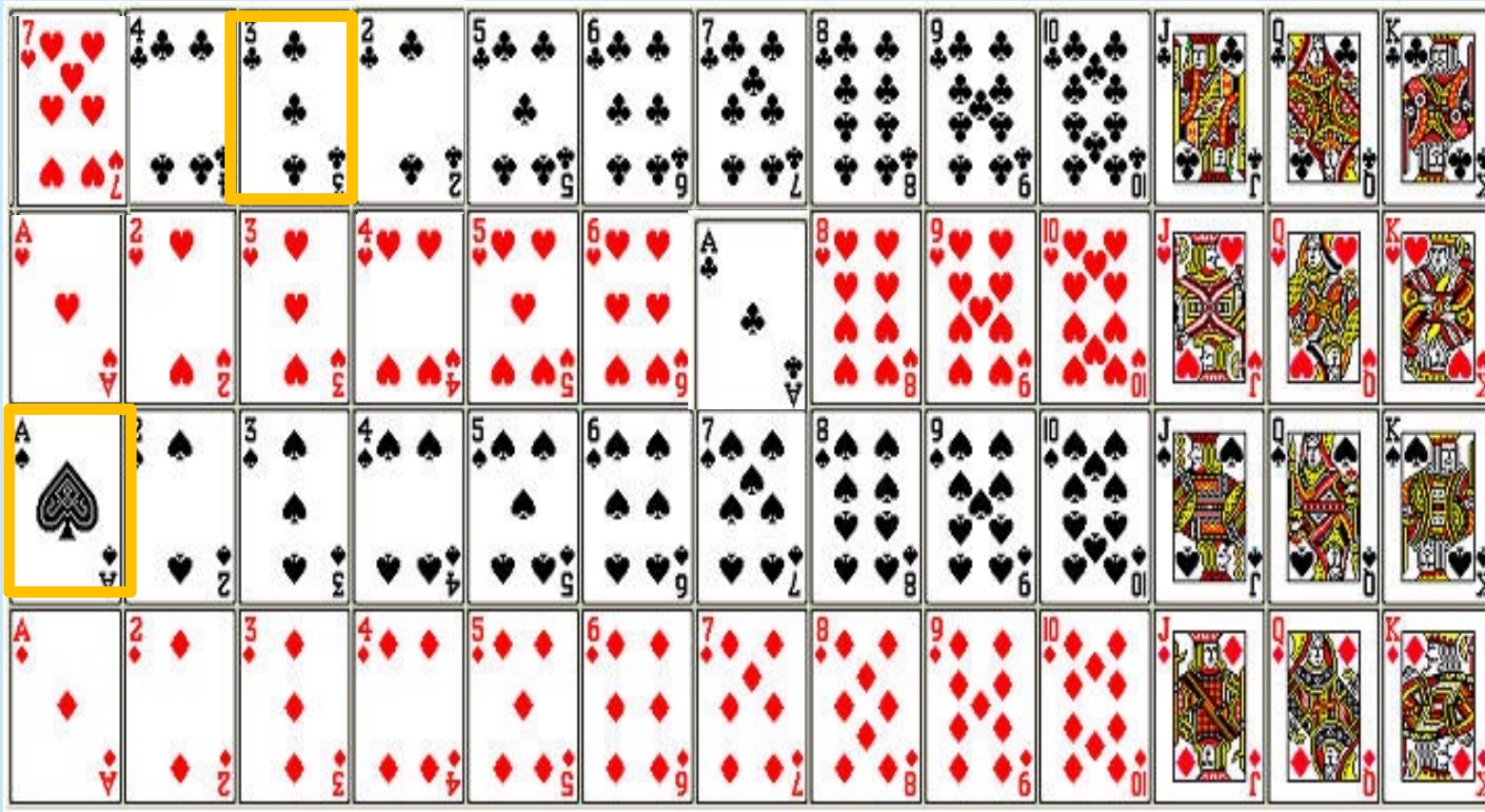
Παράδειγμα: Το ανακάτεμα της τράπουλας



Παράδειγμα: Το ανακάτεμα της τράπουλας



Παράδειγμα: Το ανακάτεμα της τράπουλας



```
int N = deck.length;
for (int i = 0; i < N; i++) {
    int r =           ;
    String t = deck[r];
    deck[r] = deck[i];
    deck[i] = t;
}
```

εναλλαγή

Τυχαίο ανάμεσα σε i και N-1

N=52

```

int N = deck.length;
for (int i = 0; i < N; i++) {
    int r = i + (int) (Math.random() * (N-i));
    String t = deck[r];
    deck[r] = deck[i];
    deck[i] = t;
}

```

εναλλαγή

← τυχαίο ανάμεσα σε i και N-1

N=52

i	(int) (Math.random()*(N-i))	i + (int) (Math.random() * (N-i))
0	0..51	0..51
1	0..50	1..51
2	0..49	2..51
	...	
	...	
50	0..1	50..51
51	0..0	51..51

Βάζουμε τα διάφορα τμήματα μαζί για να συνθέσουμε ολόκληρο το πρόγραμμα

```
public class Deck {  
    public static void main(String[] args) {  
        String[] suit = { "Clubs", "Diamonds", "Hearts", "Spades" };  
        String[] rank = { "2", "3", "4", "5", "6", "7", "8", "9",  
                           "10", "Jack", "Queen", "King", "Ace" };  
  
        int SUITS = suit.length;  
        int RANKS = rank.length;  
        int N = SUITS * RANKS;  
  
        String[] deck = new String[N];  
        for (int i = 0; i < RANKS; i++)  
            for (int j = 0; j < SUITS; j++)  
                deck[SUITS*i + j] = rank[i] + " of " + suit[j];  
  
        for (int i = 0; i < N; i++) {  
            int r = i + (int) (Math.random() * (N-i));  
            String t = deck[r];  
            deck[r] = deck[i];  
            deck[i] = t;  
        }  
  
        for (int i = 0; i < N; i++)  
            System.out.println(deck[i]);  
    }  
}
```

αποφυγή "hardwired" σταθερών

κτίσιμο τράπουλας

ανακάτεμα τράπουλας

εκτύπωση ανακατεμένης τράπουλας

Δοκιμή προγράμματος για το ανακάτεμα της τράπουλας

```
% java Deck  
5 of Clubs  
Jack of Hearts  
9 of Spades  
10 of Spades  
9 of Clubs  
7 of Spades  
6 of Diamonds  
7 of Hearts  
7 of Clubs  
4 of Spades  
Queen of Diamonds  
10 of Hearts  
5 of Diamonds  
Jack of Clubs  
Ace of Hearts  
...  
5 of Spades
```

```
% java Deck  
10 of Diamonds  
King of Spades  
2 of Spades  
3 of Clubs  
4 of Spades  
Queen of Clubs  
2 of Hearts  
7 of Diamonds  
6 of Spades  
Queen of Spades  
3 of Spades  
Jack of Diamonds  
6 of Diamonds  
8 of Spades  
9 of Diamonds  
...  
10 of Spades
```

Εισάγοντας Άρθρωση και Επαναχρησιμοποίηση

```
public class Deck{

    public static String[] createDeck(){
        String[] suit = { "Clubs", "Diamonds", "Hearts", "Spades" };
        String[] rank = { "2", "3", "4", "5", "6", "7", "8", "9",
                           "10", "Jack", "Queen", "King", "Ace" };
        int SUITS = suit.length;
        int RANKS = rank.length;
        int N = SUITS * RANKS;

        String[] deck = new String[N];
        for (int i = 0; i < RANKS; i++)
            for (int j = 0; j < SUITS; j++)
                deck[SUITS*i + j] = rank[i] + " of " + suit[j];

        return deck;
    }

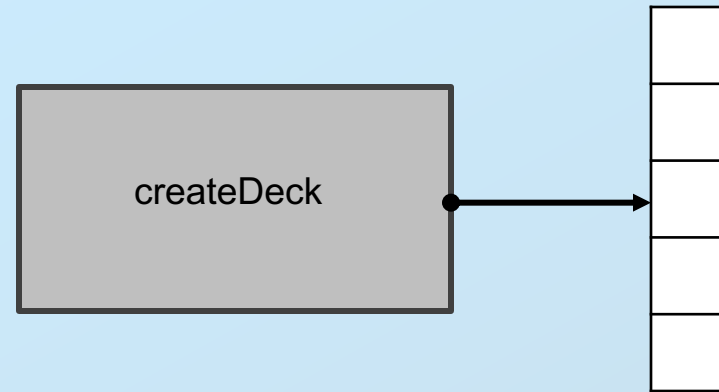
    public static void shuffleDeck(String[] deck){
        int N = deck.length;
        for (int i = 0; i < N; i++) {
            int r = i + (int) (Math.random() * (N-i));
            String t = deck[r];
            deck[r] = deck[i];
            deck[i] = t;
        }
    }
}
```

```
// συνέχεια ...

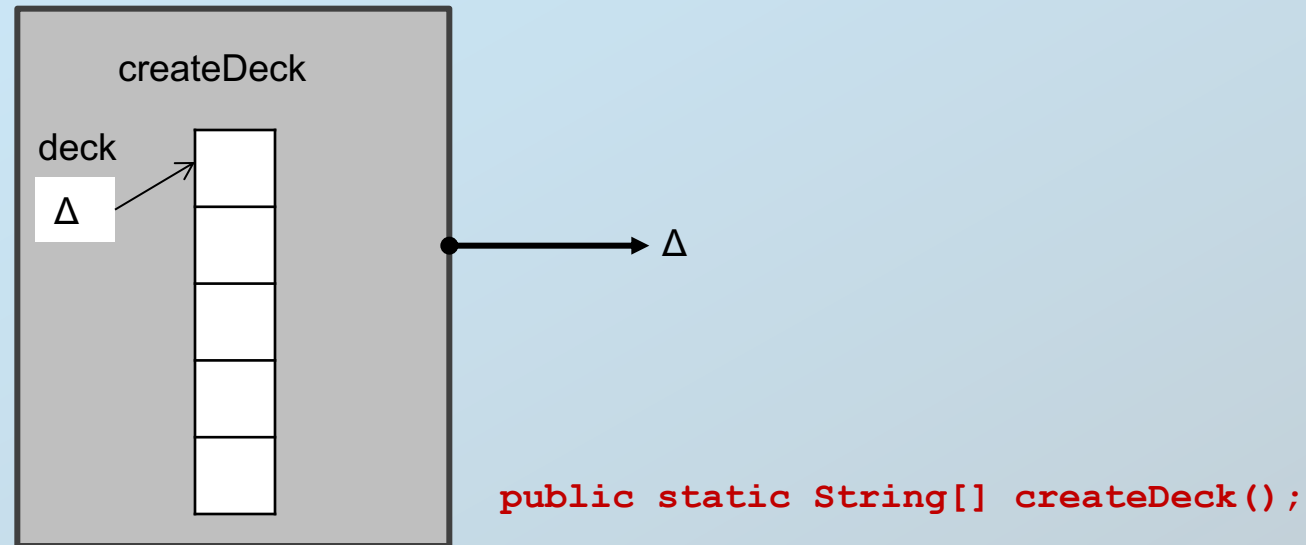
public static void displayDeck(String[] deck){
    for (String s : deck)          // Νέα μορφή for -
        System.out.println(s);    // διέρχεται των στοιχείων του πίνακα
    }

public static void main(String[] args){
    String[] deck = createDeck();   // Δημιουργία
    shuffleDeck(deck);              // Επεξεργασία
    displayDeck(deck);              // Παρουσίαση
    }
}
```

Διεπαφές Συναρτήσεων (Ταυτότητες Συναρτήσεων – signatures)



Ένας πίνακας ταυτίζεται με τη διεύθυνση του, δηλαδή τη διεύθυνση του πρώτου στοιχείου του. Ουσιαστικά, η createDeck δημιουργεί τον πίνακα deck και επιστρέφει τη διεύθυνσή του.



Διεπαφές Συναρτήσεων

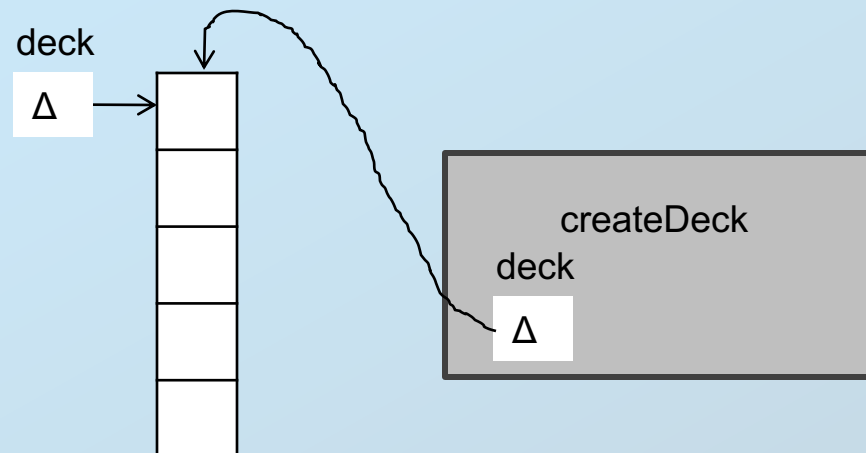
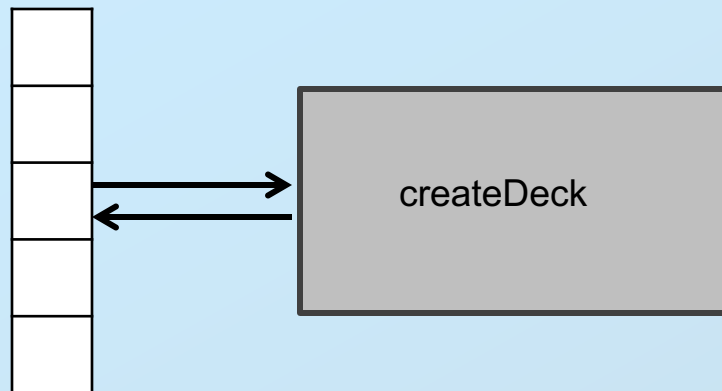
Εναλλακτικά η createDeck θα μπορούσε να λάβει ένα (περίγραμμα) πίνακα και να το γέμιζε – διαφορετική διεπαφή:

```
public static void createDeck(String[] deck){
    String[] suit = { "Clubs", "Diamonds", "Hearts", "Spades" };
    String[] rank = { "2", "3", "4", "5", "6", "7", "8", "9",
                     "10", "Jack", "Queen", "King", "Ace" };
    int SUITS = suit.length;
    int RANKS = rank.length;

    for (int i = 0; i < RANKS; i++)
        for (int j = 0; j < SUITS; j++)
            deck[SUITS*i + j] = rank[i] + " of " + suit[j];
}
```

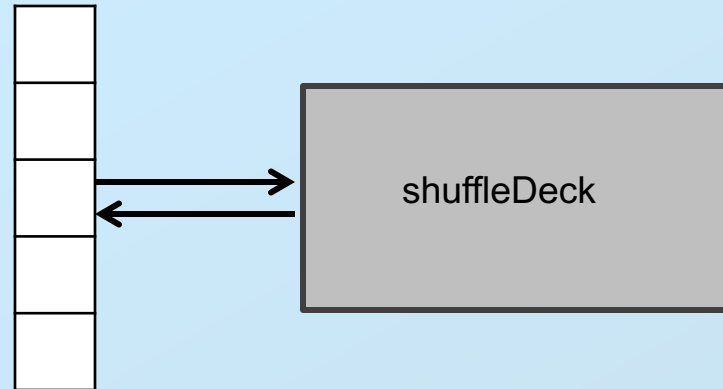
```
public static void main(String[] args){
    String[] deck = new String[52];           // Κενός πίνακας - περίγραμμα
    createDeck(deck);                         // Γέμισμα πίνακα
    shuffleDeck(deck);                       // Επεξεργασία
    displayDeck(deck);                       // Παρουσίαση
}
```

Διεπαφές Συναρτήσεων

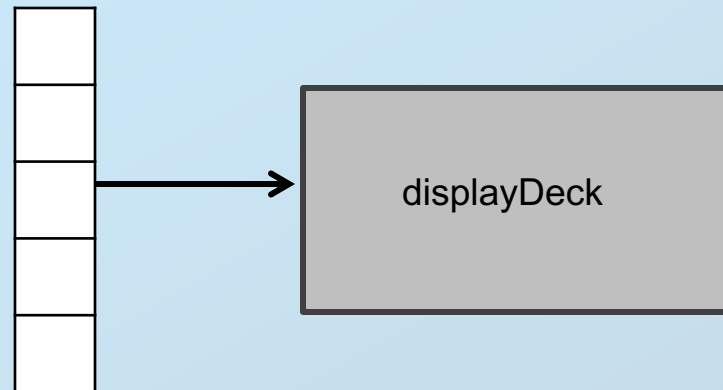


```
public static void createDeck(String[] deck);
```

Διεπαφές Συναρτήσεων



```
public static void shuffleDeck(String[] deck);
```

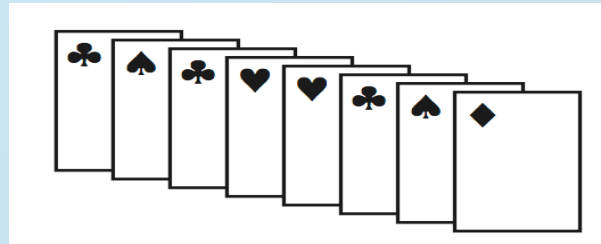


```
public static void displayDeck(String[] deck);
```

Παράδειγμα: Ο Συλλέκτης Κουπονιών

Με δεδομένο τον αριθμό N των διαφορετικών ειδών καρτών, πόσες κάρτες θα πρέπει να συλλέξεις μέχρι να έχεις τουλάχιστον μία κάρτα από το κάθε είδος;

Θεωρούμε ότι για κάθε κάρτα που συλλέγεται υπάρχει η ίδια πιθανότητα σε σχέση με τα διαφορετικά είδη.



Αλγόριθμος Προσομοίωσης

Επανειλημμένα επέλεξε ένα ακέραιο αριθμό, i , μεταξύ 0 και $N-1$. Σταμάτησε όταν έχεις τουλάχιστον μία κάρτα από το κάθε είδος.

Βασικό Υπο-πρόβλημα: Πώς γνωρίζουμε ότι έχουμε ήδη βρει μια κάρτα είδους **i**;

Χρησιμοποιούμε έναν **boolean** πίνακα, έτσι ώστε **found[i]** να γίνεται **true**, όταν έχουμε συλλέξει μια κάρτα είδους **i**. Όλες οι είσοδοι του πίνακα **found** αρχικοποιούνται σε **false**. Η αρχικοποίηση γίνεται αυτόματα.

Συλλέκτης Κουπονιών: Υλοποίηση του Προγράμματος

```
public class CouponCollector {
    public static void main(String[] args) {
        int N = Integer.parseInt(args[0]);
        int cardcnt = 0;    // number of cards collected
        int valcnt = 0;     // number of distinct cards

        // do simulation
        boolean[] found = new boolean[N];
        while (valcnt < N) {
            int val = (int) (Math.random() * N);
            cardcnt++;
            if (!found[val]) {
                valcnt++;
                found[val] = true;
            }
        }

        // all N distinct cards found
        System.out.println(cardcnt);
    }
}
```

↖
είδος επόμενης κάρτας
(ανάμεσα σε 0 και N-1)

Αποσφαλμάτωση Προγράμματος

Προσθέστε κώδικα για να εκτυπώνονται τα περιεχόμενα όλων των μεταβλητών (ίχνος προγράμματος)

val	found						valcnt	cardcnt
	0	1	2	3	4	5		
	F	F	F	F	F	F	0	0
2	F	F	T	F	F	F	1	1
0	T	F	T	F	F	F	2	2
4	T	F	T	F	T	F	3	3
0	T	F	T	F	T	F	3	4
1	T	T	T	F	T	F	4	5
2	T	T	T	F	T	F	4	6
5	T	T	T	F	T	T	5	7
0	T	T	T	F	T	T	5	8
1	T	T	T	F	T	T	5	9
3	T	T	T	T	T	T	6	10

Με πίνακες απαιτείται η ιχνηλάτηση πολλών μεταβλητών!

Συλλέκτης Κουπονιών: Εναλλακτική Υλοποίηση του Προγράμματος

```
public class CouponCollector {
    private static boolean allCollected(int[] cs){
        for (int x : cs)
            if (x == 0) return false;
        return true;
    }

    private static int sum (int[] xs){
        int s = 0;
        for (int x : xs) s += x;
        return s;
    }

    public static void main(String[] args) {
        int N = Integer.parseInt(args[0]);
        int[] cards = new int[N];
        for (int i = 0; i < N; i++) cards[i] = 0;
        while (!allCollected(cards)) {
            cards[(int) (Math.random() * N)]++;
        }
        System.out.println(sum(cards));
    }
}
```

Συλλέκτης Κουπονιών: Μαθηματικό Πλαίσιο

Με δεδομένο τον αριθμό N των διαφορετικών ειδών καρτών, πόσες κάρτες θα πρέπει να συλλέξεις μέχρι να έχεις τουλάχιστο μία κάρτα από το κάθε είδος;

Περίπου

$$1 \quad N/N +$$

$$2 \quad N/(N-1) +$$

$$3 \quad N/(N-2) +$$

$$4 \quad N/(N-3) +$$

...

$$N-1 \quad N/2 +$$

$$N \quad N/1 =$$

$$N (1 + 1/2 + 1/3 + \dots + 1/N)$$

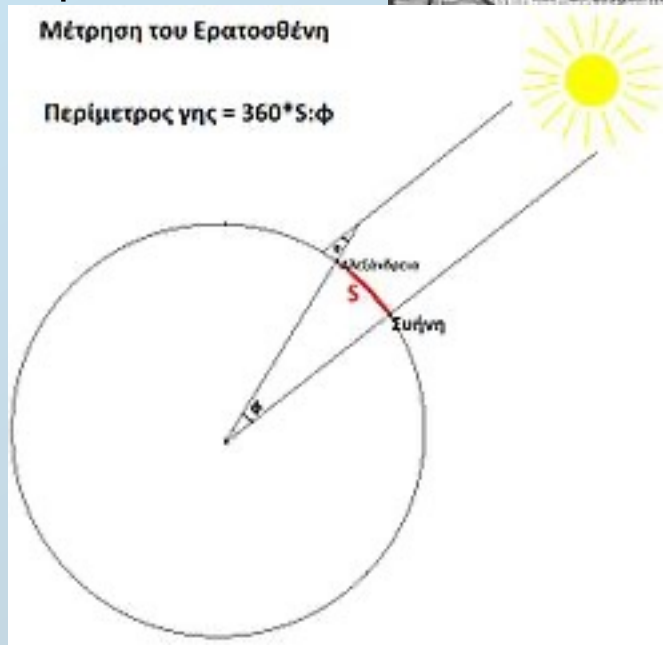
$$\sim N \ln N$$

Ερατοσθένης ο Κυρηναίος

- Κυρήνη, 276 π.Χ. – Αλεξάνδρεια, 194 π.Χ.
- Αρχαίος Έλληνας μαθηματικός, γεωγράφος, αστρονόμος, γεωδαίτης, μουσικός, ποιητής, ιστορικός, φιλόλογος και συγγραφέας
- Θεωρείται ο πρώτος άνθρωπος στην ιστορία που υπολόγισε το μέγεθος της Γης και κατασκεύασε ένα σύστημα συντεταγμένων με παράλληλους και μεσημβρινούς
- Κατασκεύασε και έναν χάρτη του κόσμου, όπως τον θεωρούσε



Μέτρηση περιφέρειας γης
τον 3 π.Χ. αιώνα



Παράδειγμα: Το κόσκινο του Ερατοσθένη

- Πρώτος αριθμός (prime number): Ένας ακέραιος > 1 του οποίου οι μόνοι θετικοί παράγοντες είναι το 1 και ο εαυτός του
 - Π.χ., 2, 3, 5, 7, 11, 13, 17, 23, ...
- Συνάρτηση που μετρά το πλήθος των πρώτων αριθμών:
$$\pi(N) = \# \text{ primes } \leq N$$
 - Π.χ., $\pi(17) = 7$
- Το κόσκινο του Ερατοσθένη
 - Στον πίνακα `isPrime[]` καταγράφονται οι ακέραιοι που είναι πρώτοι αριθμοί
 - Επανέλαβε για $i=2$ μέχρι $\sqrt{N}$
 - Εάν ο αριθμός i δεν είναι σημειωμένος ως πρώτος αριθμός
 - Ο i δεν είναι πρώτος αριθμός αφού έχει ήδη βρεθεί κάποιος παράγοντάς του
 - Εάν ο αριθμός i είναι σημειωμένος ως πρώτος αριθμός
 - Ο i είναι πρώτος αριθμός αφού δεν έχει μικρότερους παράγοντες
 - Σημείωσε όλους τους πολλαπλάσιους αριθμούς του i ως μη πρώτους αριθμούς

Πώς καταγράφονται οι πρώτοι αριθμοί $\leq N$ στον πίνακα `isPrime`

- Πρώτος αριθμός (prime number): Ένας ακέραιος > 1 του οποίου οι μόνοι θετικοί παράγοντες είναι το 1 και ο εαυτός του
 - Π.χ., 2, 3, 5, 7, 11, 13, 17, 23, ...
- Συνάρτηση που μετρά το πλήθος των πρώτων αριθμών:
$$\pi(N) = \# \text{ primes } \leq N$$
 - Π.χ., $\pi(17) = 7$

i	isPrime																								
	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	
	T	T	T	T	T	T	T	T	T	T	T	T	T	T	T	T	T	T	T	T	T	T	T	T	
2	T	T	F	T	F	T	F	T	F	T	F	T	F	T	F	T	F	T	F	T	F	T	F	T	
3	T	T	F	T	F	T	F	F	F	T	F	T	F	F	F	T	F	T	F	F	F	T	F	T	
5	T	T	F	T	F	T	F	F	F	T	F	T	F	F	F	T	F	T	F	F	F	T	F	F	
	T	T	F	T	F	T	F	F	F	T	F	T	F	F	F	T	F	T	F	F	F	T	F	F	

Υλοποίηση αλγόριθμου “Κόσκινο του Ερατοσθένη”

```
public class PrimeSieve {  
    public static void main(String[] args) {  
        int N = Integer.parseInt(args[0]);
```

```
        // initially assume all integers are prime  
        boolean[] isPrime = new boolean[N+1];  
        for (int i = 2; i <= N; i++)  
            isPrime[i] = true;
```

```
        // mark non-primes <= N using Sieve of Eratosthenes  
        for (int i = 2; i*i <= N; i++) {  
            if (isPrime[i]) {  
                for (int j = i; i*j <= N; j++)  
                    isPrime[i*j] = false;  
            }  
        }
```

εάν ο i είναι πρώτος αρ, σημείωσε
τα πολλαπλάσια του i ως μη
πρώτους αριθμούς

```
        // count primes  
        int primes = 0;  
        for (int i = 2; i <= N; i++)  
            if (isPrime[i]) primes++;  
        StdOut.println("The number of primes <= " + N + " is " + primes);
```

```
    }  
}
```

```
% java PrimeSieve 1000  
The number of primes <= 1000 is 168
```

Επέκταση της κλάσης **PrimeSieve** με την προσθήκη της μεθόδου **displayPrimes** για την εκτύπωση των πρώτων αριθμών, ανά 10 αριθμούς σε κάθε γραμμή

```
$ java PrimeSieve 1000
The number of primes <= 1000 is 168
Listing the primes <= 1000
2  3  5  7  11  13  17  19  23  29
31 37 41 43 47 53 59 61 67 71
73 79 83 89 97 101 103 107 109 113
127 131 137 139 149 151 157 163 167 173
179 181 191 193 197 199 211 223 227 229
233 239 241 251 257 263 269 271 277 281
283 293 307 311 313 317 331 337 347 349
353 359 367 373 379 383 389 397 401 409
419 421 431 433 439 443 449 457 461 463
467 479 487 491 499 503 509 521 523 541
547 557 563 569 571 577 587 593 599 601
607 613 617 619 631 641 643 647 653 659
661 673 677 683 691 701 709 719 727 733
739 743 751 757 761 769 773 787 797 809
811 821 823 827 829 839 853 857 859 863
877 881 883 887 907 911 919 929 937 941
947 953 967 971 977 983 991 997
```

Εκτύπωση των 168 πρώτων
αριθμών ανά 10 σε κάθε γραμμή



```
public static void displayPrimes (boolean isPrime[], int n){
    System.out.println("Listing the primes <= " + (isPrime.length-1));
    int count = 0;
    for (int i = 2; i < isPrime.length; i++){
        if (isPrime[i]) {System.out.print(i + " "); count++;
            if (count % n == 0) System.out.println();}
    }
}
```

```
// calling displayPrimes from main
displayPrimes(isPrime,10);
```

Είναι ο ακόλουθος αναδιατυπωμένος ορισμός της displayPrimes λογικά ισοδύναμος με τον πιο πάνω ορισμό;

```
public static void displayPrimes (boolean isPrime[], int n){
    System.out.println("Listing the primes <= " + (isPrime.length-1));
    int count = 0;
    for (int i = 2; i < isPrime.length; i++){
        if (count % n == 0) System.out.println();
        if (isPrime[i]) {System.out.print(i + " "); count++;}
    }
}
```

Παράδειγμα: Γραμμική Αναζήτηση (Linear Search)

Πίνακας x

x[0]	x[1]	x[2]	x[3]	x[4]	x[5]	x[6]	x[7]	x[8]
6	2	7	1	3	9	4	10	12

↑
pos

Αρχίζοντας από το πρώτο στοιχείο του πίνακα, διαδοχικά συγκρίνουμε το υπό αναζήτηση στοιχείο με το στοιχείο του πίνακα, μέχρις ότου είτε να εντοπισθεί το στοιχείο, είτε να εξετασθούν όλα τα στοιχεία

Επομένως, η αλγοριθμική δομή είναι επανάληψη

```
public class LinearSearch {  
    public static void main (String[] args) {  
        int[] x = {6, 2, 7, 1, 3, 9, 4, 10, 12};  
  
        int pos;  
  
        int item = Integer.parseInt(args[0]);  
  
        for (pos=0; pos < x.length && item != x[pos];++pos); //empty body  
  
        if (pos == x.length)  
            System.out.println("Item " + item + " not found");  
        else  
            System.out.println("Item " + item + " found");  
    }  
}
```

Προσοχή: Σειρά συνθηκών

```
item != x[pos] && pos < x.length
```

```
public class LinearSearch {  
    public static void main (String[] args) {  
        int[] x = {6, 2, 7, 1, 3, 9, 4, 10, 12};  
  
        int pos;  
  
        int item = Integer.parseInt(args[0]);  
  
        for (pos=0; pos < x.length;++pos)  
            if (item == x[pos])  
                break;  
  
        if (pos == x.length)  
            System.out.println("Item not found");  
        else  
            System.out.println("Item found");  
    }  
}
```

Στην χειρότερη περίπτωση πόσες επαναλήψεις χρειάζονται αν ο πίνακας έχει μέγεθος N ;
 N

break

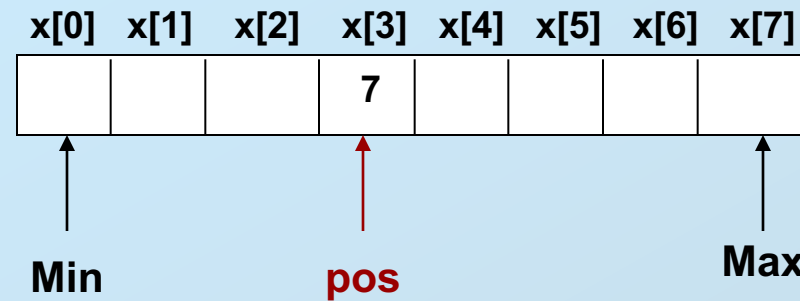
- Η εντολή `break` τερματίζει άμεσα την εντολή επανάληψης που την περιέχει
- Επιτρέπει την διακοπή μιας επανάληψης πριν γίνει `false` έλεγχο της συνθήκης επανάληψης

```
int i;  
for (i=2; i*i<=N; ++i)  
    if (N%i==0) break;  
if (i*i>N)  
    System.out.println(N + " is prime");
```

i	N
2	21
3	

Παράδειγμα: Δυαδική αναζήτηση

Τα στοιχεία του πίνακα είναι ταξινομημένα σε αύξουσα σειρά
(υποθέστε πίνακα ακέραιων και ψάχνω το 13)



Αλγόριθμος;

```
int[] table; // table with data
int item;    // item we search for

int Min = 0, Max = table.length - 1, pos;

while(Min <= Max) {
    pos = (Min + Max) / 2;
    if (item > table[pos])
        Min = pos + 1;
    else if (item < table[pos])
        Max = pos - 1;
    else{
        // found in position pos
        break;
    }
}
// if Min>Max not found
```

partial code

Πόσα βήματα

$N/2$	1
$N/4$	2
$N/8$	3
$N/16$	4
..	
..	
1	x

Πόσα βήματα

$N/2$	1
$N/4$	2
$N/8$	3
$N/16$	4
..	
..	
$1=N/N$	x

Πόσα βήματα

$N/2$

1

$N/4$

2

$N/8$

3

$N/16$

4

..

..

$$1 = N/N = N/2^x,$$

$$2^x = N,$$

$$x = \log_2 N$$

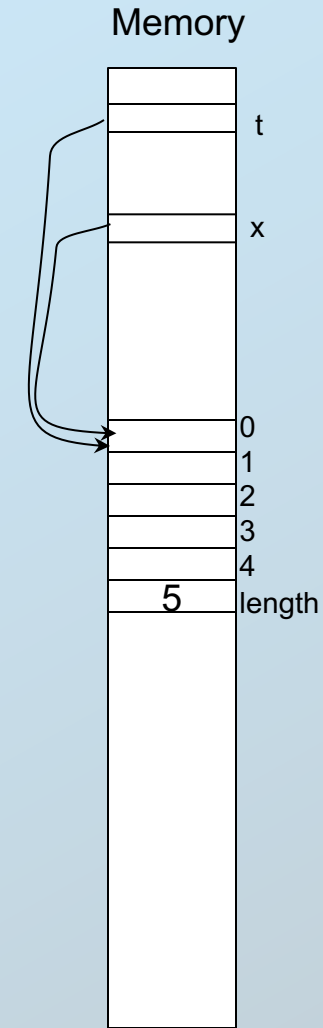
Μεταβλητές Πινάκων είναι δείκτες

```
public class Main{  
    public static void main(String[] args){  
        int[] t = new int[5];  
        int[] x = t;  
  
        for(int i=0; i<t.length; ++i)  
            System.out.println(i + " : " + t[i] + " " + x[i]);  
        x[3] = 996;  
        System.out.println();  
        for(int i=0;i<t.length;++i)  
            System.out.println(i + " : " + t[i] + " " + x[i]);  
        System.out.println(x==t + " " + x.length == t.length);  
    }  
}
```

```
0:0 0  
1:0 0  
2:0 0  
3:0 0  
4:0 0  
  
0:0 0  
1:0 0  
2:0 0  
3:996 996  
4:0 0  
  
true true
```

Αναπαράσταση στην Μνήμη

```
public class Main{  
    public static void main(String[] args){  
        int[] t = new int[5];  
        int[] x=t;  
  
        for(int i=0; i<t.length; ++i)  
            System.out.println(i + " : " + t[i] + " " + x[i]);  
        x[3]=996;  
        System.out.println();  
        for(int i=0; i<t.length; ++i)  
            System.out.println(i + " : " + t[i] + " " + x[i]);  
        System.out.println(x==t + " " + x.length == t.length);  
    }  
}
```



Σύνοψη

- Πίνακας ως δομή δεδομένων
 - Η Java παρέχει ειδική στήριξη
- Δημιουργία Πίνακα
 - Δήλωση, υλοποίηση, αρχικοποίηση
 - Τυχαία προσπέλαση
- Επεξεργασία Πίνακα
 - Αναφορά σε στοιχεία του πίνακα
 - Κατάλληλη αλγοριθμική δομή: επανάληψη

Στο βιβλίο

- **Ενότητα 1.4**